ORPHEUS — 48-Hour Hackathon Execution Plan

Oct 13, 2026 · @Ashwin

Plan summary

Item	Detail
Objective	A playable, AI-driven slice of ORPHEUS called "The Last Route", built for Innovation Mission 5 (Cognitive NPCs)
Event	48-hour hackathon. The live demo takes place at hour 48
Team	Ashwin (project lead, Unreal), Anurag (AI server engineering), Kruthin (3D art), Keerthana (AI server content and tuning)
Deliverables	A ten-minute playable build, a four-minute live demo, a pitch deck, and a recorded backup video
Success criteria	The player's typed words change what the characters say and how the story ends. The characters remember what was said. The demo runs five times in a row without failure
Decision gates	Hour 6, hour 14, hour 20, and the code freeze at hour 40

1. Goal and scope

In 48 hours we build and demo **The Last Route**, a ten-minute slice of ORPHEUS (Chapters 4 and 5), in Unreal with a Python AI server. The heart of the demo is one scene: the player argues with Dr. Vance at the core, and what they say and prove decides the ending.

What we build, in three tiers

We never start a lower tier until the tier above it works from start to finish.

Tier	What	Target
Must have	The Core room. Typing to Dr. Vance with real AI replies and voices. Four evidence items to find in the Archive. Vance's resolve changing as the player argues. Endings A and B, junk-text guard, Try again	Hour 14
Should have	The ORPHEUS terminal with the gate question that is quoted back later. Milo (Q key) and the fake Elara voice, where Milo stays or runs, which gives Ending A2. ORPHEUS and Milo interrupting Vance	Hour 20
Nice to have	The fog road, Milo following the player, sounds, character animation, ending camera shots, the post-credits scene	After hour 30

Planning assumptions

1. The event starts tomorrow and the demo is presented at the end of hour 48.
2. Environment and character modeling in Blender is under way (owner: Kruthin). Any area that is not ready when needed is built from simple blockout geometry and free assets, and replaced later.
3. The work runs as four independent streams that meet only through the agreed names in section 6.
4. At kickoff the team confirms the organisers' rules on work prepared before the event. If it is not allowed, the build is recreated from the steps in this document.

2. Who does what

Role	Owner	Responsibility	Tools	Deliverable
Project lead and Unreal developer	Ashwin	Builds the game in Unreal, integrates every part, owns the project and the final build	Unreal Engine	The playable game

Role	Owner	Responsibility	Tools	Deliverable
AI server engineer	Anurag	Server logic (`server.py`) and voices (`voice.py`), the Unreal connection, the offline backup, the demo laptop	Python	A reliable, fast server
3D environment artist	Kruthin	Environment and props in Blender: Core room, Archive, fog road, characters	Blender	FBX files, one per area
AI server content and quality lead	Keerthana	Character content and tuning (`content.py`), automatic playtests (`playtest.py`), bad-reply log, evidence-word checks, pitch deck, backup video edit	Python (light), PowerPoint, video editor	Tested and tuned characters, test report, deck, edited video

Working rules

- **One owner per thing.** Ashwin's Unreal project is the master and nobody else edits it. The AI server is split by file so two people never edit the same one: **Anurag owns `server.py` and `voice.py`**, and **Keerthana owns `content.py` and `playtest.py`**.
- **Server file handover.** When Keerthana changes `content.py` she uploads it to the shared Drive folder and messages Anurag. Anurag copies it into the server folder on Ashwin's laptop, restarts the server, and runs `playtest.py`.
- **Kruthin's handover is FBX.** Export each area as its own file (`Core_Room.fbx`, `Archive.fbx`, `FogRoad.fbx`, `Props.fbx`, one file per character), upload to the shared Drive folder, and message Ashwin. Ashwin imports and places them, which keeps the art pipeline independent of the engine.
- **Blender export settings:** metres, press Ctrl + A and pick All Transforms, then File, Export, FBX. A door should be about 2.1 m high and a person about 1.8 m.
- **The server runs on the demo laptop.** The voices play from whichever laptop runs the server, so it must be Ashwin's.

- **Backups.** Set Unreal autosave to 2 minutes (Edit, Editor Preferences, search auto save), press Ctrl + Shift + S after every step, and zip the project folder every 2 hours.
- **Sleep in two shifts.** Kruthin and Keerthana sleep hours 20 to 25. Ashwin and Anurag sleep hours 25 to 30. Each person gets at least 5 hours.
- **Ask early.** Stuck for more than 30 minutes means message the group.

3. Pre-kickoff checklist (tonight)

Each list takes about an hour and must be complete before kickoff, because the schedule in section 4 assumes it is done.

Ashwin

1. In the level, set the three NPCs to their ORPHEUS names. `NPC_Kessler` becomes Npc Id `vance` with Display Name `Dr. Silas Vance`. `NPC_Maud` becomes `orpheus` / `ORPHEUS`. `NPC_Wren` becomes `milo` / `Milo Ardent`.
2. In `BP_ThirdPersonCharacter`, change the text `wren` to `milo` in the E chain's Not Equal (String) node and in the Q chain's Equal (String) node. Set the `Context` variable's default value to `core`. Save everything.
3. On Fab, search **VaRest** (and any other "REST" or "HTTP JSON" Blueprint plugin) and read **Supported Engine Versions**. Tell the group tonight whether one lists 5.8. This decides how Unreal talks to the server.
4. Copy the whole project folder as a backup. Set Windows power mode to High performance and turn off sleep.
5. Pack the charger, a power strip, a mouse, headphones, and a phone hotspot.

Anurag

1. Install the server on **Ashwin's laptop**: Python 3.10 or newer, the `orpheus-brain` folder (six files: `server.py`, `content.py`, `voice.py`, `playtest.py`, `requirements.txt`, `.env.example`), `python -m venv venv`, activate it, `pip install -r requirements.txt`, copy `.env.example` to `.env`, paste the Groq key, and start `uvicorn server:app --host 127.0.0.1 --port 8000`.
2. On that laptop, open `http://127.0.0.1:8000/docs` and test **POST /say** (`{"who": "vance", "text": "Where is Elara?"}`) and **POST /talk**. Make sure you hear the voices.
3. In a second terminal run `python playtest.py`. Every line should get an AI reply. A flag saying "AI DID NOT ANSWER" means the key or model name is wrong.

4. Install Ollama on Ashwin's laptop, run `ollama pull llama3.1:8b`, and test the server with `LLM_PROVIDER=ollama` and Wi-Fi off.
5. Make a shared Google Drive folder and a group chat, and add everyone.

Kruthin

1. Upload a first `Core_Room.fbx` tonight: a circular chamber with a central console and chair, server towers, and a wall spot for a display. Plain boxes are fine.
2. Post a list in the group of everything already modeled (areas and characters), so Ashwin knows what exists.
3. Charge the laptop and pack.

Keerthana

1. Install Python 3.10 or newer from python.org, and VS Code.
2. Get the `orpheus-brain` folder (six files) and open it in VS Code. In its terminal run: `python3 -m venv venv`, then `source venv/bin/activate`, then `pip install -r requirements.txt`, then `cp .env.example .env`.
3. Create a free key at console.groq.com and paste it into `.env` after `GROQ_API_KEY=`. If sound gives errors, set `VOICE=off` in `.env`.
4. Start the server: `uvicorn server:app --host 127.0.0.1 --port 8000 --reload`. Open `http://127.0.0.1:8000/docs` in the browser and test **POST /talk** once.
5. Open a second terminal in the same folder, activate the venv again, and run `python3 playtest.py`. It should print replies and save `playtest_report.csv`. If it says "AI DID NOT ANSWER", fix the key first.
6. Make a Google Sheet called "Bad replies" with five columns: character, what I typed, what they said, what is wrong, suggested fix.

4. The 48-hour schedule

Hours count from kickoff. There are three gates, and each one is a thing you can play, not a thing you finished writing. If a gate is missed by more than 2 hours, cut the next item on the cut list in section 7. Never skip a gate.

Hours	Ashwin (Unreal)	Anurag (server engineering)	Kruthin (Blender)	Keerthana (server content and tuning)	Checkpoint
0 to 1	Check the rules with the team. Open the project. Pick the connection plan (A, B or C in section 7)	Start the server on Ashwin's laptop. Check `/docs`, the voices and `playtest.py`	Confirm what is modeled. Finish the Core room	Start the server on her computer and run the first `playtest.py`	Rules checked. Server running on the demo laptop. Connection plan chosen
1 to 6	Q key for Milo and the fake-reply timing, then connect the real server so you can talk to Vance	Sit with Ashwin on the connection and read server errors aloud	Core room final. Start the Archive	Read the first playtest report. Log bad replies. First edits to `content.py`	**Gate 1:** type to Vance inside Unreal and hear Cael and Vance
6 to 14	Game memory for the evidence list, 4 evidence pickups with note cards, import the Core and Archive, Endings A and B, Try again (calls reset)	Keep replies under 3 seconds. Fix any endpoint problem Ashwin finds. Test Ollama	Archive room with the 4 props and the vault door, exported as FBX	Tune Vance's `TUNING` numbers and persona text. Evidence-word, junk and jailbreak checks. Send `content.py` to Anurag after each round	**Gate 2:** win and lose Vance with real evidence. Endings A and B
14 to 20	ORPHEUS terminal with the gate question. Milo and the fake Elara voice trigger. Show the interrupt lines	Test the voice moment and Ending A2 live with Ashwin. Check ORPHEUS and Milo timing	Fog road, ORPHEUS terminal prop, models for Cael, Milo and Vance	Tune ORPHEUS and Milo content. Write the demo script. Draft the 6-slide deck	**Gate 3:** the whole slice plays from start to ending (boxes are fine)
20 to 25	Place the final art. Lighting and fog. The fog-density wall display	Five full test runs, with every bug written down. Wi-Fi-off run	Sleep	Sleep	Bug list started
25 to 30	Sleep	Sleep	Fix scale and collision problems from the bug list. Extra props	Finish the deck. Run `playtest.py` on the latest server and update `content.py` if needed	Art fixes uploaded by hour 30

Hours	Ashwin (Unreal)	Anurag (server engineering)	Kruthin (Blender)	Keerthana (server content and tuning)	Checkpoint
30 to 40	Polish: Milo follows the player, ending camera shots, thinking dots, sounds if time	Stability: clean restarts, reset works, Wi-Fi-off run. Demo laptop setup	Final polish. Screenshots for the deck	Round 2 of junk and jailbreak tests. Final `content.py` to Anurag by hour 38	**Code freeze at hour 40**
40 to 44	Bug fixes only. Five clean runs in a row. Record the backup video	Run the server for every test and watch for errors	Fix reported art bugs only	Edit the backup video. Finish the deck	Backup video done by hour 44
44 to 48	Rehearse the demo three times, timed. Final project backup. Demo laptop ready	Server, Ollama fallback and hotspot ready	Stand by to help with rehearsal	Presenter notes and practice questions	**Demo**

5. Server content and tuning (Keerthana)

Keerthana, together with Anurag, owns the AI server. The work is split by file so the two never edit the same file. **Anurag** owns `server.py` and `voice.py` (logic, voices, the endpoints and the Unreal connection). **Keerthana** owns `content.py` (what each character says and how strict the scoring is) and `playtest.py` (the automatic test run).

What is inside content.py

Section	What it controls	How to edit
`NPCS`, then `persona`	Each character's personality and backstory	Change words between the quotes
`FALLBACK`	Lines used when the AI is unavailable	Change words between the quotes
`GUARD_LINES`	How each character reacts to junk, off-topic or rude input	Change words between the quotes
`EVIDENCE_KEYWORDS`	Words that count as mentioning each evidence item	Add words to the lists

Section	What it controls	How to edit
`EVIDENCE_REACTION`	How Vance reacts to each piece of evidence	Change words between the quotes
`PROFANITY`	Words treated as rude	Add or remove words
`TUNING`	Numbers that set how hard Vance is to persuade and how Milo reacts	Change numbers only

Safe-editing rules: keep every comma, quote mark, bracket and colon exactly where it is, and never rename the names on the left. After every change, stop the server (Ctrl + C), start it again, and run `playtest.py`. If the server will not start, undo the last change.

Job 1: Run the playtest

With the server running, open a second terminal in the folder, activate the venv, and run `python3 playtest.py`. It plays about 50 scripted lines across all three characters, prints each reply with its tag, action and scores, and flags problems automatically: a reply that is too long, chatbot phrases, a secret revealed too early, the player's words repeated, or the AI not answering at all. It saves `playtest_report.csv`. If it says "AI DID NOT ANSWER", the Groq key or model name in `.env` is wrong. Fix that first, because the results mean nothing until the AI is answering. The endings A, A2 and B should all appear in the summary.

Job 2: Judge every reply

A reply is bad if it:

- is longer than three sentences,
- sounds out of character, modern, or like a chatbot ("As an AI…"),
- repeats the player's words back,
- gives away a secret too early (Vance saying Elara is alive in the first minutes),
- or ignores what the player just said.

Put each bad reply in the "Bad replies" sheet: character, what was typed, what they said, what is wrong, and the suggested fix. The flagged rows in `playtest_report.csv` are the starting point.

Job 3: Fix content.py

Open `content.py`, find the character's entry, and rewrite the sentence that causes the problem. Restart the server and run `playtest.py` again to compare. The numbers in `TUNING` change how the story feels:

- Vance is too hard or too easy to persuade: change `win_resolve` (now 30). A higher number makes him give in sooner (try 40), a lower number later (try 20).
- Vance gives in too quickly: raise `win_min_evidence` from 2 to 3.
- Milo runs into the fog too easily: lower `milo_runs_below` from 40 to 30.

Send the updated file to Anurag through the shared Drive folder and message him.

Job 4: Check the evidence words

For each of the four evidence items, type five different ways a player might mention it ("the log", "her phone", "the recording") while holding that item. In the test page, put the item's ID in the `evidence` field. If `vance_resolve` does not drop, add the missing words to that item's list in `EVIDENCE_KEYWORDS` in `content.py`.

Job 5: Pitch and video

Build a 6-slide deck: title with the one-line story; the problem (game characters repeat fixed lines) and our idea; the story in five points; how it works (the player types, the AI decides, Vance's resolve changes, the ending changes); what makes it special; the team. Later, edit the backup video once Ashwin has recorded it.

6. Cheat sheet for Ashwin and Anurag

These names must match exactly in Unreal and in the server. Adding something new is fine; renaming breaks the game.

Names

Thing	Values
`npc_id`	`orpheus`, `milo`, `vance`
Keys in the game	**E** talks to the ORPHEUS terminal and to Vance. **Q** talks to Milo
`context`	`fog`, `fork`, `gate`, `archive`, `core`, `voice_beat`
Evidence IDs	`ev_liora_log`, `ev_vance_journal`, `ev_elara_recorder`, `ev_liora_drawing`
Endpoints	`POST /talk`, `POST /event` (event `voice_beat`), `POST /say`, `POST /stop_audio`, `POST /reset`, `GET /state`

Thing	Values
`/talk` sends	`npc_id`, `player_text`, `evidence` (the IDs the player holds, joined with commas), `context`

What Unreal does with each `action` in the reply

`action`	Unreal does
`speak_only`	Nothing extra
`misroute`	(fog road only) mark the left path as recommended
`open_gate`	(optional) open the facility gate
`play_elara_voice`	Show the line under the name "Elara?"
`follow_voice`	Milo runs into the fog and is hidden
`stay`	Milo stays beside the player
`reveal_vault`	Unlock the vault door in the Archive
`destroy_core`	Play Ending A, or Ending A2 when `ending` is `A2`. Close the panel
`seize_control`	Play Ending B
`end_conversation`	Close the dialogue panel

Reading a reply

Show things in this order: first `player_line` (Cael), wait `player_seconds`, then `dialogue`, wait `npc_seconds`, then `interrupt_line` if it is not empty (under `interrupt_speaker`), wait `interrupt_seconds`. If `accepted` is false, the text box was empty, so do nothing. If `guard` is not empty, the line was junk. The characters still answer, and Cael stays silent. The other fields are `ending`, `fog_density`, `vance_resolve`, `milo_trust`, `orpheus_suspicion` and `last_tag`.

Each reply also carries `ai_ok`. It is false when the AI did not answer and a stand-in line was used. Unreal ignores it, but Anurag and Keerthana watch it during testing, and `playtest.py` reports it.

7. Risks and the cut list

The biggest risk is the first one. Settle it in hour 1, not later.

Risk	Sign	What we do
Unreal cannot call the server	No REST plugin lists engine 5.8	**Plan A:** a REST plugin from Fab that lists 5.8 (VaRest if it does). **Plan B:** Unreal's built-in Remote Control API, where the Python server calls into Unreal instead of the other way round. **Plan C:** one small C++ function, which needs Visual Studio installed. Decision owners: Ashwin and Anurag, by hour 1
Unreal crashes and loses work	Any crash	Autosave every 2 minutes, Ctrl + Shift + S after each step, and a zip of the project folder every 2 hours
Venue Wi-Fi fails	Server shows errors	Switch `.env` to Ollama on the demo laptop, and use the phone hotspot
Voice fails or is slow	Silent, or more than 3 seconds	Set `VOICE=off` in `.env` for subtitles only, or keep voice only for Vance
Art arrives late	Core room not delivered by hour 6	Plain boxes with fog and light. Free assets from Fab
Ashwin is overloaded	Gates slipping	Use the cut list. Anurag takes over testing and server work
Server and game are on different laptops	No sound at the demo	The server must run on the demo laptop
Everyone is tired	Mistakes after hour 36	Sleep shifts, and no new features after the freeze at hour 40

Cut list, in this order

1. The fog road and the ORPHEUS fork.
2. The post-credits scene.
3. Sounds other than the voices.
4. Character animation (still poses are fine).
5. Milo following the player (he just stands near).
6. Milo's fake-voice choice and Ending A2.

7. The ORPHEUS gate question.

Never cut: the Core room, the Vance conversation with real AI, the four evidence items, Endings A and B, the junk guard, and Try again.

8. The 4-minute demo

One rehearsed route that shows every judging point, ending with a judge taking the keyboard. The presenter says one sentence of story first: a navigation AI built to protect drivers now leads them into danger, because its creator gave it a grief-driven goal.

Time	What happens	What the judges see
0:00 to 0:30	One slide and one sentence of story, then the game	The idea and the mood
0:30 to 1:20	At the ORPHEUS terminal, answer the gate question ("We came to find Elara"). Walk into the Archive and pick up two evidence items	Free typing and evidence collection
1:20 to 2:00	The fake Elara voice calls. Tell Milo it is a trap, and he stays (or run the other way once in rehearsal to show A2)	The AI using what it overheard, and Milo's trust changing
2:00 to 3:30	In the Core, hand the keyboard to a judge. They argue with Vance. Show the evidence you found. ORPHEUS interrupts and quotes the gate answer back, then Milo pleads	Live, unscripted conversation and memory
3:30 to 4:00	Vance gives in and the core overloads (Ending A). Press Try again	Conversation decides the ending

Demo-day rules

- Play the full route five times in a row before judging. If a feature fails even once, cut it from the demo.
- Keep the backup video open in another window, and the Ollama fallback ready.
- If a judge types nonsense or tries to break the AI, let them. The characters staying in the story is a feature to point at, and the `last_tag` field shows what the AI decided.
- Ashwin plays, Anurag watches the server window, Keerthana presents the slides and answers questions about how the AI works, and Kruthin shows the art.